Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

CS101

Counters, score tables, and grouping

Today
Welcome to your next lesson: Counters, score tables, and grouping.

I'm preparing your personalized lesson based on what you've learned, what's coming up, and how to build your skills step by step.

Let's keep moving forward — every step counts.

Imagine you're watching a basketball game 🏀

You start with a simple counter:

score = 0
Every time your team scores 2 points: score = score + 2

That works for one team. But what if you want to track both teams at once (or all players' individual points)?

A single number isn't enough — you need labeled counters like drawers:

"Lakers" → 56
"Celtics" → 60

That's where a dictionary counter comes in.

Quick check before we dive into code:

If you want to track how many times each word appears in a text, does that sound like:

one single counter, or
many labeled counters (one per word)?

You can just answer: "single counter" or "many labeled counters."

many labeled counters

Exactly — many labeled counters 👍

Each label (word) needs its own count. A Python dictionary is perfect for this:

key = the word (label)
value = how many times we've seen it

Visually:

Word (key)	Count (value)
"the"	5
"cat"	2
"dog"	3

Next step, I want to show you the core counting pattern with a simple example like colors in a list.

Are you okay if we go step by step through that pattern now? (yes/no)

Yes, let's go

Nice & Let's build it step by step with a real example.

Say we have a list of colors from shirts people are wearing:

Python

```
colors = ["red", "blue", "red", "green", "blue", "red"]
```

We want a result like:

Python

```
{"red": 3, "blue": 2, "green": 1}
```

The core counting pattern in Python looks like this:

Python

```
counts = {}

for color in colors:
    counts[color] = counts.get(color, 0) + 1
```

Idea in words, one loop at a time:

Look at the current color.
Get its current count from counts (or use 0 if it's not there yet).
Add 1.
Store it back in the dictionary.
Does this step-by-step idea make sense so far, before we trace it with a tiny example?

Yes, it makes sense

What do you think the dictionary counts looks like after that loop? Pick an option below 👇

You have this code:

```
colors = ["red", "blue", "red"]
counts = {}

for color in colors:
    counts[color] = counts.get(color, 0) + 1
```

After the loop finishes, what is the value of counts?

A.

```
{"red": 1, "blue": 1}
```

B.

{"red": 2, "blue": 1}

C.

{"red": 3, "blue": 0}

D.

{"red": 1, "blue": 2}

Nice work, Ron — you nailed it 🎯

For this code:

Python

```
colors = ["red", "blue", "red"]
counts = {}

for color in colors:
    counts[color] = counts.get(color, 0) + 1
```

we end up with:

Python

{"red": 2, "blue": 1}
Quick trace:

Start: counts = {}
First "red" → counts.get("red", 0) is 0 → store 1 → {"red": 1}
"blue" → counts.get("blue", 0) is 0 → store 1 → {"red": 1, "blue": 1}
Second "red" → counts.get("red", 0) is now 1 → store 2 → {"red": 2, "blue": 1}
Now I want you to drive the pattern in the editor.

In the code editor, please:

Create a list:
animals = ["cat", "dog", "cat", "bird", "dog", "cat"]
Use the same counting pattern to build counts for animals.
print(counts) at the end.

Run it first, then hit Submit code so I can see it.

After you submit, I'll help you check if the counts look right 🐱🐶🐦

Python

```
animals = ["cat", "dog", "cat", "bird", "dog", "cat"]
counts = {}

for animal_type in animals:
    counts[animal_type] = counts.get(animal_type, 0) + 1
print(counts)
```

That's perfect, Ron 🎉

Your output:

Text

{'cat': 3, 'dog': 2, 'bird': 1}
matches the list exactly:

"cat" appears 3 times

"dog" appears 2 times
"bird" appears 1 time

And your code uses the exact counting pattern I wanted:

Python

```
counts[animal_type] = counts.get(animal_type, 0) + 1
```

Next, we'll tweak this pattern slightly to do totals instead of counts (same skeleton, different value).

Ready to switch from "how many?" to "how much?" in the next step? (yes/no)

Yes, I'm ready

Awesome 😄 Let's go step by step.

We'll keep the same shape as counting, but now we'll track totals.

Imagine you have deliveries for each driver:

Python

```
trips = [
    ("Alice", 10),   # 10 miles
    ("Bob", 5),
    ("Alice", 7),
    ("Bob", 3),
]
```

Goal: total miles per driver:

Python

```
{"Alice": 17, "Bob": 8}
```

Pattern (very close to counting):

Python

```
totals = {}

for name, miles in trips:
    totals[name] = totals.get(name, 0) + miles
```

Key idea:

For counting we did: + 1
For totals we do: + miles (or price, score, etc.)
Does this link clearly in your head:

counting pattern = add 1 each time,

accumulation pattern = add some value each time (like miles)?

Yes, that makes sense

Great 👍 Let's lock in the accumulation pattern with one clear example.

Imagine a tiny "store log":

Python

```
sales = [
    ("Alice", 12.50),
    ("Bob", 5.00),
    ("Alice", 3.50),
]
```

We want: total money per person:

Python

```
{"Alice": 16.0, "Bob": 5.0}
```

The pattern is:

Python

```
totals = {}

for name, amount in sales:
    totals[name] = totals.get(name, 0) + amount
```

Step-by-step for "Alice" on her second sale:

totals.get("Alice", 0) → current total (16.0? 12.5? 0?)
Add amount (the new sale)
Store it back for that same name.
I'd like you to try this pattern yourself next in the editor.

Ready to do a small sales totals exercise with your own code?

Yes, I'm ready

Nice, let's make it concrete 💰

Step by step task for you in the code editor:

Create this list:

```
sales = [
    ("Alice", 12.50),
    ("Bob", 5.00),
    ("Alice", 3.50),
    ("Bob", 4.00),
]
```

Make an empty dict: totals = {}

Loop over sales and use the accumulation pattern:

```
for name, amount in sales:
    totals[name] = totals.get(name, 0) + amount
```

print(totals) at the end.

Run it first, then hit Submit code so I can see the result.

I'll help you check if Alice and Bob's totals look right.

Python

```
sales = {
    ("Alice", 12.50),
    ("Bob", 5.00),
    ("Alice", 3.50),
    ("Bob", 4.00),
}
totals = {}

for name, amount in sales:
    totals[name] = totals.get(name, 0) + amount
print(totals)
```

Nice job, Ron — your logic is spot on 🎯

You used the right accumulation pattern:

Python

```
for name, amount in sales:
    totals[name] = totals.get(name, 0) + amount
```

and your result:

Python

```
{'Bob': 9.0, 'Alice': 16.0}
```

is exactly right:

Bob: 5.00 + 4.00 = 9.0
Alice: 12.50 + 3.50 = 16.0

One small thing: you used a set for sales instead of a list:

Python

```
sales = {
    ("Alice", 12.50),
    ("Bob", 5.00),
    ("Alice", 3.50),
    ("Bob", 4.00),
}
```

In Python, { ... } with comma-separated items (no key: value) is a set, and sets:

don’t keep order
don’t keep duplicates (so one of Alice’s or Bob’s entries could be dropped)

For this pattern, we usually want a list:

Python

```
sales = [
    ("Alice", 12.50),
    ("Bob", 5.00),
    ("Alice", 3.50),
    ("Bob", 4.00),
]
```

Quick check-in:

Does the difference between using { ... } (set) and [...] (list) here make sense, and why we prefer a list for this kind of loop?

yes

Great ☑

You’ve now seen two patterns:

Counting – + 1 each time
Accumulation – + value each time (like amount)

There’s one more key pattern for this lesson: grouping — where each key maps to a list of things instead of a number.

Example idea: group students by grade level:

Python

```
{
    "freshman": ["Alice", "Tom"],
    "senior": ["Bob"],
```

}
Here, instead of counting or totaling, we're collecting items together under each key.

Before I show you the grouping pattern in code, does the idea of "dictionary where each key has a list of items" feel clear?

Yes, that's clear

Nice 👍 then we can go straight to the pattern.

Think of subjects in school:

Python

```
subjects = [
    ("math", "Alice"),
    ("english", "Bob"),
    ("math", "Tom"),
    ("history", "Alice"),
]
```

Goal (group by subject):

Python

```
{
    "math": ["Alice", "Tom"],
    "english": ["Bob"],
    "history": ["Alice"]
}
```

Core grouping pattern:

Python

```
groups = {}

for subject, student in subjects:
    if subject not in groups:
        groups[subject] = []          # start empty list for this key
    groups[subject].append(student)   # then add to the list
```

Step-by-step idea:

If this key hasn't been seen → create an empty list.
Then append the new item to that list.

So: instead of + 1 or + value, we do .append(...) to a list.

Does this "initialize empty list, then append" flow make sense before we try one in code?

Yes, it makes sense

Great, let's have you use that pattern yourself 😊

In the code editor, please:

Create this list:

```
pets = [
    ("Alice", "cat"),
    ("Bob", "dog"),
    ("Alice", "hamster"),
    ("Bob", "fish"),
    ("Alice", "dog"),
]
```

Make an empty dict: owners = {}

Loop over pets and group by owner name using the pattern:

If the owner name is not in owners yet, set it to an empty list.
Then append the pet to that owner's list.
print(owners) at the end.

Run it, then hit Submit code so I can see it. I'll help you check that each person has the right list of pets 🐾

Python

```
pets = [
    ("Alice", "cat"),
    ("Bob", "dog"),
    ("Alice", "hamster"),
    ("Bob", "fish"),
    ("Alice", "dog"),
]
owners = {}

for name, pet in pets:
    if name not in owners:
        owners[name] = []          # start empty list for this key
    owners[name].append(pet)
print(owners)
```

Beautifully done, Ron 👏

Your output:

Python

```
{'Alice': ['cat', 'hamster', 'dog'], 'Bob': ['dog', 'fish']}
```

matches the data perfectly, and you used the grouping pattern exactly right:

Python

```
if name not in owners:
    owners[name] = []      # initialize list
owners[name].append(pet)   # then append
```

So now you've successfully used:

Counting (per-key + 1)
Accumulation (per-key + value)
Grouping (per-key list + .append)
I want to quickly check that the difference between these is solid in your head.

Which of these patterns would you use for this question:

"For each game level, which players reached that level?"

You can answer in words like: counting, accumulation, or grouping.

grouping , players by condition reeached level

Exactly right — grouping is the right choice here 💯

"For each game level, which players reached that level?"

You’re not asking:

“How many players?” (that would be counting)
or “How many total points?” (that would be accumulation)

You’re asking “which players”, so you need a list of players per level, just like your owners → pets example.

So it would look like:

Python

```
{
    1: ["Alice", "Bob"],
    2: ["Alice"],
    3: ["Charlie"]
}
```

Now I want to quickly test that you can choose the right pattern from a few scenarios.

I’ll give you a small multiple-choice question next to check that, okay?

Sounds good!

What do you think fits best here? Pick an option above 👆

For each product category, you want to know the total revenue (sum of all sales amounts) for that category. Which dictionary pattern should you use?

A.

Counting pattern (add 1 for each item)

B.

Accumulation pattern (add the sale amount each time)

C.

Grouping pattern (append each sale to a list)

Nice pick, Ron — that’s exactly the right pattern 🎯

Total revenue per category = add the sale amount each time, not just 1 and not building lists you don’t need.

So the accumulation pattern fits:

Python

```
totals[category] = totals.get(category, 0) + amount
```

Counting: would only tell you how many sales, not how much money.
Grouping: would store every sale in a list, then you’d still have to sum later.
Accumulation: goes straight to the final total you care about.
At this point you’ve:

Used counting, accumulation, and grouping in real code ☑
Correctly chosen which pattern to use from a scenario ☑

You’ve basically mastered the core patterns for this lesson.

Do you feel ready to wrap up this lesson and move on to the next one, or would you like one more practice challenge with these patterns first?

I'm ready to move on

Love it. You earned this move up 🚀

Quick recap of what you can now do:

Use a dictionary to count things:

counts[key] = counts.get(key, 0) + 1
Use a dictionary to sum totals per key:

totals[key] = totals.get(key, 0) + amount
Use a dictionary to group items into lists:

if key not in groups:
    groups[key] = []
groups[key].append(item)
You're thinking in solid patterns now — that's a big shift toward writing cleaner, smarter code, not just "making it work."

When you're ready, go ahead and jump into the next lesson in Learn.
If anything in this one feels fuzzy later, come back and I'll walk through it with you again, step by step.

```
animals = ["cat", "dog", "cat", "bird", "dog", "cat"]
counts = {}

for animal_type in animals:
    counts[animal_type] = counts.get(animal_type, 0) + 1
print(counts)

sales = [
    ("Alice", 12.50),
    ("Bob", 5.00),
    ("Alice", 3.50),
    ("Bob", 4.00),
]
totals = {}

for name, amount in sales:
    totals[name] = totals.get(name, 0) + amount
print(totals)

pets = [
    ("Alice", "cat"),
    ("Bob", "dog"),
    ("Alice", "hamster"),
    ("Bob", "fish"),
    ("Alice", "dog"),
]
owners = {}

for name, pet in pets:
    if name not in owners:
        owners[name] = []          # start empty list for this key
    owners[name].append(pet)
print(owners)
```